

# Hoja de trabajo #5 — Orquestación de sistemas multiagente

CC3116 · Caso Parachute S.A. · Arquitecturas centralizada, jerárquica y descentralizada · Respuestas a las preguntas planteadas

---

## 1. Resumen de lo implementado

Se construyeron tres programas que resuelven exactamente el mismo problema —responder FAQs, consultar Open-Meteo para la zona de aterrizaje (14.013722, -90.771611), aplicar los criterios de seguridad de salto y calendarizar la cita— cambiando únicamente la forma de orquestar a los agentes:

- **centralizada.py** — un orquestador único con los cuatro especialistas expuestos como herramientas mediante `as_tool()`.
- **jerarquica.py** — tres niveles: un director, dos gerencias intermedias (Información y Operaciones) y cuatro especialistas; `as_tool()` anidado.
- **descentralizada.py** — cinco agentes pares que se ceden el control con handoffs; no hay coordinador.

Las tres comparten el paquete `core/`: el cliente de Open-Meteo, el motor de reglas de seguridad, el repositorio de citas, la base de FAQs y el catálogo de `function_tool`. Por eso la lógica de las integraciones es idéntica en las tres implementaciones y ninguna arquitectura la reescribe: lo único que cambia es *quién* puede llamar a *qué* y en qué orden.

Decisión de diseño importante: los umbrales de seguridad (viento  $< 20$  /  $20-28$  /  $> 28$  km/h, ráfagas  $> 35$  km/h, precipitación  $> 0.0$  mm, nubosidad  $< 30$  /  $30-75$  /  $> 75$  %) **no los decide el modelo de lenguaje**. Viven en `core/safety.py` como código determinístico que el agente solo invoca. El LLM interpreta la intención del usuario y comunica el resultado; la decisión que implica riesgo físico es auditable y reproducible.

## 2. ¿Qué arquitectura resuelve mejor este problema? ¿Por qué?

**Respuesta corta:** para el alcance actual gana la **centralizada**; para el alcance que Parachute S.A. anunció ("va a seguir incrementando los requerimientos funcionales") la elección correcta es la **jerárquica**. La **descentralizada es la peor opción** en este caso concreto.

### 2.1 Por qué la centralizada es la mejor hoy

- **El problema tiene un orden obligatorio.** Consultar clima  $\rightarrow$  dictaminar seguridad  $\rightarrow$  agendar es una secuencia con dependencia de datos: el paso  $n$  necesita la salida del paso  $n-1$ . Un orquestador que llama a los especialistas como funciones impone ese orden por construcción.
- **Hay una regla dura que no se puede negociar.** Si el veredicto es PROHIBIDO no se agenda. Con un solo punto de decisión esa condición se verifica una vez y siempre en el mismo lugar (además de estar respaldada en la propia herramienta de agenda).
- **El alcance es pequeño.** Cuatro capacidades y unas ocho herramientas caben sin problema en el contexto de un solo coordinador, sin degradar la selección de herramienta.

- **Es la más barata y la más rápida.** Menos saltos entre agentes que la jerárquica: menos latencia acumulada y menos tokens por consulta.
- **Una sola voz.** El cliente recibe una respuesta integrada y coherente; nunca percibe que detrás hay varios agentes.

## 2.2 Por qué la jerárquica es la mejor apuesta a mediano plazo

- El cliente **advirtió explícitamente** que seguirá agregando requerimientos (pagos, transporte, inventario de equipo, CRM, video...). Un orquestador plano con veinte o treinta herramientas se degrada: el prompt crece, las descripciones compiten entre sí y aumentan los errores de selección.
- Agrupar por gerencias mantiene el **abanico de decisión pequeño en cada nivel** (2 a 5 opciones). Agregar una capacidad nueva se vuelve local: se añade un especialista bajo la gerencia que corresponde y el director ni se entera.
- Permite **separación de funciones**: la gerencia que informa no es la misma que autoriza y ejecuta. En un negocio donde una decisión equivocada implica riesgo físico, eso es valioso.
- Su costo es real y hay que reconocerlo: más saltos, más latencia, más tokens y el riesgo de "teléfono descompuesto" al resumir en cada nivel. Con el alcance actual sería sobreingeniería.

## 2.3 Por qué la descentralizada no encaja aquí

- **Nadie garantiza el orden.** El handoff cede el control por completo; que se respete la secuencia clima → seguridad → agenda depende de que cada agente obedezca su prompt. Es una garantía probabilística donde se necesita una determinística.
- **El agente que recibe el control le habla al cliente.** Puede responder con información parcial o contradecir lo que dijo el par anterior, y no hay quien consolide.
- **Es la más difícil de auditar y probar.** El camino que tomó una conversación no es fijo, y ante un incidente de seguridad hay que reconstruirlo.
- Sería la opción correcta en otro escenario: dominios independientes, sin orden obligatorio y con conversaciones largas dentro de una misma especialidad (por ejemplo, soporte técnico dividido por producto).

## 2.4 Comparación

| Criterio                                      | Centralizada                        | Jerárquica                  | Descentralizada                      |
|-----------------------------------------------|-------------------------------------|-----------------------------|--------------------------------------|
| Garantiza el orden clima → seguridad → agenda | Sí, por construcción                | Sí, dentro de cada gerencia | No, depende del prompt               |
| Latencia y costo por consulta                 | Bajo                                | Medio-alto                  | Bajo                                 |
| Escala al crecer los requerimientos           | Se satura el prompt del orquestador | Buena: crecimiento local    | Buena en nodos, mala en coordinación |
| Coherencia de la respuesta final              | Alta: una sola voz                  | Alta                        | Variable                             |
| Trazabilidad / auditoría                      | Alta                                | Media                       | Baja                                 |
| Ajuste al caso Parachute S.A.                 | Mejor hoy                           | Mejor a mediano plazo       | No recomendada                       |

Conclusión práctica: se recomienda operar con la centralizada y migrar a la jerárquica cuando el catálogo de capacidades supere aproximadamente ocho o diez herramientas. Gracias a que las integraciones están abstraídas en core/, esa migración cuesta reconectar agentes, no reescribir lógica.

### 3. ¿Es necesario usar un sistema multiagente en este caso? ¿Por qué?

**Respuesta corta: no, todavía no es necesario.** El problema tal como está planteado se resuelve bien con un solo agente que tenga las mismas herramientas. El sistema multiagente se justifica por lo que viene, no por lo que hay.

#### 3.1 Argumentos de que un solo agente basta hoy

- Son **ocho herramientas** con propósitos claramente distintos. Los modelos actuales seleccionan bien con ese volumen; el problema de selección aparece bastante más arriba.
- **La parte crítica no necesita un LLM.** Los umbrales son comparaciones numéricas: es un `if`, no un razonamiento. Tener un "oficial de seguridad" con personalidad propia no hace el veredicto más correcto que llamar a la función.
- Los cuatro roles **no tienen conocimientos ni estilos incompatibles** que obliguen a separarlos en prompts distintos; caben cómodamente en uno solo.
- Cada agente extra cuesta: una llamada más al modelo, más latencia, más tokens y un punto adicional donde la información se resume mal.

#### 3.2 Cuándo sí se vuelve necesario

- **Crecimiento del catálogo.** El propio cliente lo anunció. Al pasar de ocho a treinta herramientas, la descripción de todas deja de caber útilmente en un prompt y la tasa de error en la selección sube. Ahí la especialización deja de ser estética y pasa a ser funcional.
- **Aislamiento de permisos.** Que el agente de FAQs no tenga forma de escribir en la agenda ni de cancelar citas es una garantía estructural, no una instrucción que se pueda ignorar.
- **Costo y modelos heterogéneos.** Responder FAQs puede hacerlo un modelo pequeño y barato; coordinar puede requerir uno más capaz. Separar agentes permite asignar modelo por rol.
- **Control de contexto.** Cada especialista ve solo lo que necesita, lo que reduce tokens y reduce la superficie de errores y de inyección de instrucciones.
- **Mantenimiento por equipos.** Distintas personas pueden mantener distintos agentes sin pisarse un único prompt gigante.

**Criterio para decidir:** conviene un sistema multiagente cuando el costo de coordinación (latencia, tokens, resúmenes intermedios, complejidad) es menor que el beneficio de la especialización (precisión en la selección de herramientas, aislamiento de permisos, escalabilidad del mantenimiento). En el estado actual de Parachute S.A. ese punto de equilibrio aún no se cruza; con dos o tres requerimientos más, sí. Implementarlo ahora con las integraciones abstraídas es una inversión razonable, siempre que se reconozca que hoy se está pagando un sobre costo de latencia a cambio de esa flexibilidad futura.

### 4. Cómo se logró la abstracción pedida

| Módulo              | Responsabilidad                                                                                                             | Qué evita                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| core/weather.py     | Única parte que conoce Open-Meteo; normaliza la respuesta a un objeto <code>ClimaDia</code> y valida la ventana de 16 días. | Que cada arquitectura arme su propia URL o parsee el JSON.                    |
| core/safety.py      | Motor determinístico de reglas; devuelve un Veredicto con nivel y justificación por criterio.                               | Que el LLM improvise umbrales o los interprete distinto en cada arquitectura. |
| core/scheduling.py  | Repositorio de citas y validación de cupos; rechaza agendar con veredicto PROHIBIDO.                                        | Que la regla de negocio dependa solo del prompt.                              |
| core/tools.py       | Catálogo único de <code>function_tool</code> , agrupado por especialidad.                                                   | Duplicar definiciones de herramientas entre los tres programas.               |
| core/agents_base.py | Fábrica de los cuatro especialistas, idénticos en las tres arquitecturas.                                                   | Reescribir prompts y herramientas por arquitectura.                           |
| core/config.py      | Proveedor de LLM y constantes del negocio.                                                                                  | Tener que tocar tres archivos al cambiar de modelo o de coordenadas.          |

Agregar el requerimiento que venga (por ejemplo, cobro con tarjeta) significa: escribir el módulo de integración, envolverlo en una `function_tool`, añadir un especialista en `agents_base.py` y conectarlo en cada arquitectura con una línea. Ninguna de las tres implementaciones cambia su lógica.